UNIVERSIDAD SERGIO ARBOLEDA

LENGUAJES DE PROGRAMACION

ACTIVIDAD ANTL4

DOCENTE

JOAQUIN FERNANDO SANCHEZ CIFUENTES

JULIAN CAMILO CARO HERRERA

Este informe se divide en 2 partes. La primera parte se encontrara la verificación de la instalación del antlr4 y el ejemplo de la presentación, la segunda parte del informe incluye la calculadora del capitulo 4

INTALACION ANTLR4

```
julian@julian-VirtualBox:~$ java org.antlr.v4.Tool
ANTLR Parser Generator  Version 4.13.2
 -o ___              specify output directory where all output is generated
 -lib ___            specify location of grammars, tokens files
 -atn                generate rule augmented transition network diagrams
 -encoding ___       specify grammar file encoding; e.g., euc-jp
 -message-format ___ specify output style for messages in antlr, gnu, vs2005
 -long-messages      show exception details when available for errors and warnings
 -listener           generate parse tree listener (default)
 -no-listener        don't generate parse tree listener
 -visitor            generate parse tree visitor
 -no-visitor         don't generate parse tree visitor (default)
 -package ___        specify a package/namespace for the generated code
 -depend             generate file dependencies
 -D<option>=value    set/override a grammar-level option
 -Werror             treat warnings as errors
 -XdbgST             launch StringTemplate visualizer on generated code
 -XdbgSTWait         wait for STViz to close before continuing
 -Xforce-atn         use the ATN simulator for all predictions
 -Xlog               dump lots of logging info to antlr-timestamp.log
 -Xexact-output-dir  all output goes into -o dir regardless of paths/package
julian@julian-VirtualBox:~$
```

FIGURA 1 (Verificacion de la instalación del antlr4)

EJEMPLO DIAPOSITIVA

Paso 1: primero se creo la gramática y se verifico que los archivos se hayan creado correctamente como se aprecia en las figuras 2 y 3

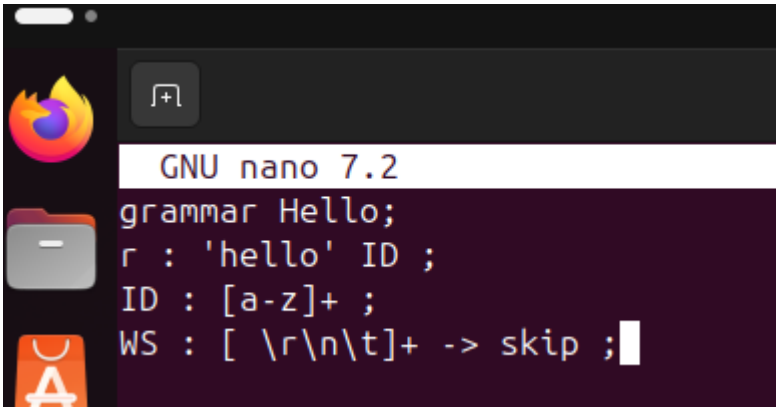

FIGURA 2 (creacion del nano con la gramatica)

```
julian@julian-VirtualBox:~/ACTIVIDADANTLR4$ nano Hello.g4
julian@julian-VirtualBox:~/ACTIVIDADANTLR4$ antlr4 Hello.g4
julian@julian-VirtualBox:~/ACTIVIDADANTLR4$ ls
HelloBaseListener.java  Hello.g4  Hello.interp  HelloLexer.interp  HelloLexer.java  HelloLexer.tokens  HelloListener.java  HelloParser.java  Hello.tokens
julian@julian-VirtualBox:~/ACTIVIDADANTLR4$
```

FIGURA 3 (Verificacion de que si se hayan creado los archivos)

PASO 2: ahora se compilara y se ejecutara ingresando como ejemplo hello Julian, Tambien se incluyo el arbol de parser. Estas dos cosas se pueden observer en la figura 4 y 5

```
julian@julian-VirtualBox:~/ACTIVIDADANTLR4$ javac Hello*.java
```

FIGURA 4 (Compilacion con javc)

```
julian@julian-VirtualBox:~/ACTIVIDADANTLR4$ grun Hello r -tokens
hello julian
[@0,0:4='hello',<'hello'>,1:0]
[@1,6:11='julian',<ID>,1:6]
[@2,13:12='<EOF>',<EOF>,2:0]
julian@julian-VirtualBox:~/ACTIVIDADANTLR4$
```

FIGURA 5(Prueba y ejecucion)

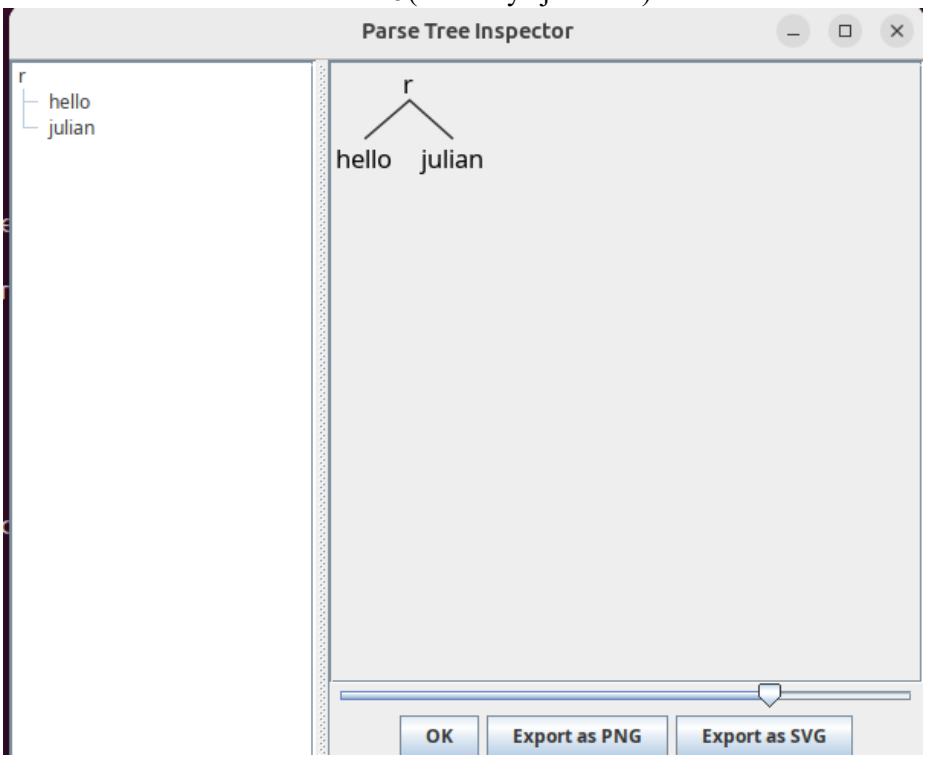

FIGURA 6(arbol grafico del parser)

CALCULADORA CAPITULO 4

1 PASO: Crear el archivo donde ira la gramatica de la calculadora, segun el libro el archive quedara con el nombre calc.g4 como se vera en la figura 7 y 8 la creacion y la gramatica.

```
julian@julian-VirtualBox:~/CALCULADORcap4$ nano calc.g4
```

FIGURA 7 (creacion del archivo)

```
  GNU nano 7.2
grammar Calc;

prog:   stat+ ;

stat:   expr NEWLINE                # printExpr
    |   ID '=' expr NEWLINE         # assign
    |   NEWLINE                     # blank
    ;

expr:   expr ('*'|'/') expr         # MulDiv
    |   expr ('+'|'-') expr         # AddSub
    |   INT                         # int
    |   ID                          # id
    |   '(' expr ')'                # parens
    ;

ID  :   [a-zA-Z]+ ;
INT :   [0-9]+ ;
NEWLINE : [\r\n]+ ;
WS  :   [ \t]+ -> skip ;
```

FIGURA8 (Gramatica)

Paso 2: ahora hay que crear el visitor y el main para que se pueda ejecutar correctamente la gramatica y asi poder hacer pruebas

Creacion del visitor

```
julian@julian-VirtualBox:~/CALCULADORcap4$ nano EvalVisitor.java
```

FIGURA 11 (Creacion del visitor)

```
  GNU nano 7.2
import java.util.HashMap;
import java.util.Map;

public class EvalVisitor extends CalcBaseVisitor<Integer> {
    Map<String, Integer> memory = new HashMap<>();

    @Override
    public Integer visitAssign(CalcParser.AssignContext ctx) {
        String id = ctx.ID().getText();
        int value = visit(ctx.expr());
        memory.put(id, value);
        return value;
    }

    @Override
    public Integer visitPrintExpr(CalcParser.PrintExprContext ctx) {
        Integer value = visit(ctx.expr());
        System.out.println(value);
        return 0;
    }

    @Override
    public Integer visitInt(CalcParser.IntContext ctx) {
        return Integer.valueOf(ctx.INT().getText());
    }

    @Override
    public Integer visitId(CalcParser.IdContext ctx) {
        String id = ctx.ID().getText();
        return memory.getOrDefault(id, 0);
    }

    @Override
    public Integer visitMulDiv(CalcParser.MulDivContext ctx) {
        int left = visit(ctx.expr(0));
        int right = visit(ctx.expr(1));
        if (ctx.op.getType() == CalcParser.MUL) return left * right;
        return left / right;
```

FIGURA 12(codigo proporciona por el libro)

Creacion del main

Ahora tocar efectuar las pruebas para eso hay que crear un archvio principal para poder ejecutar

```
julian@julian-VirtualBox:~/CALCULADORcap4$ nano CalcTest.java
```

FIGURA 13 (creacion del archivo)

```
  GNU nano 7.2
import org.antlr.v4.runtime.*;
import org.antlr.v4.runtime.tree.*;

public class CalcTest {
    public static void main(String[] args) throws Exception {
        CharStream input = CharStreams.fromStream(System.in);
        CalcLexer lexer = new CalcLexer(input);
        CommonTokenStream tokens = new CommonTokenStream(lexer);
        CalcParser parser = new CalcParser(tokens);
        ParseTree tree = parser.prog();

        EvalVisitor eval = new EvalVisitor();
        eval.visit(tree);
    }
}
```

FIGURA 14(codigo)

Creacion del archive txt con las pruebas

```
  GNU nano 7.2
a = 5
b = 2
a * b + 3
```

FIGURA 15(ENTRADA)

COMPILACION Y EJECUCION

Despues de haber creado los archivos incluidos el txt, se compilara y se ejecutara como se ve en la figura 15, como se aprecia en la figura a*b+3: seria 5*2+3 y efectivamente da 13 como se observa en la figura 16

```
julian@julian-VirtualBox:~/CALCULADORcap4$ javac -cp .:/usr/share/java/antlr4-runtime.jar *.java
julian@julian-VirtualBox:~/CALCULADORcap4$ java -cp .:/usr/share/java/antlr4-runtime.jar CalcTest < input.txt
13
```

FIGURA 16(entradas)